ShinyPLSp
User Manual

Version 1.0
June 2026

Patricio Ramírez Correa

Legal Notice and Software License

Open Source Software Agreement
Unlike proprietary commercial software, ShinyPLSp is an open-source project developed for the academic and scientific community. The source code of the software is released under the GNU General Public License (GPL) version 3.
This means you are free to use, modify, and distribute the software, provided that any derivative works are also distributed under the same open-source license. The software relies heavily on the R programming environment and open-source packages such as shiny, cSEM, and genpathmox, whose respective copyrights belong to their original authors.

Disclaimer of Warranty and Liability
The ShinyPLSp Software is provided "as is", without any warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement.
Under no circumstances shall the authors, developers, or affiliated institutions be held liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) caused by the use of or inability to use the Software.

Important Warning on Statistical Analysis
Multivariate statistical analysis software systems are inherently complex. They can sometimes yield results that are biased, mathematically distorted, or disconnected from the reality of the phenomena being modeled.
Users are strongly cautioned against blindly accepting the results provided by ShinyPLSp. You must actively double-check and validate your results against:
1. Past empirical results obtained by other means or with other software.
2. Applicable theoretical frameworks.
3. Practical, common-sense assumptions.

The software computes algorithms based on the data and parameters you provide; it does not replace the critical thinking and theoretical justification required in scientific research.

How to Cite ShinyPLSp
If you use ShinyPLSp to analyze data for a consulting project, thesis, or peer-reviewed publication, you must properly cite the software and its underlying statistical engines to acknowledge the intellectual work involved.
To cite the application:
Ramírez-Correa, P. (2026). *ShinyPLSp: An Interactive Web Application for Advanced PLS-SEM Analysis* (Version 1.0) [Computer software].
To cite the underlying estimation engines:
Users must also ensure they cite the cSEM package (Rademaker & Schuberth, 2020) for the PLS-SEM estimations, and the genpathmox package (Lamberti et al., 2016) if the PATHMOX segmentation module is used.

1. General Presentation of the Application

ShinyPLSp is a modular analytical platform designed for researchers and professionals who need to perform **Partial Least Squares Structural Equation Modeling (PLS-SEM)**. The application guides the user through a sequential workflow: loading data, preparing the database, defining the measurement model, defining the structural model, estimating, interpreting results, and, if applicable, performing advanced PATHMOX segmentation.

- **Purpose:** To estimate complex relationships between latent variables (constructs) and observable variables (indicators).
- **Target User Profile:** Researchers and data analysts needing a robust, code-free interface for PLS-SEM.
- **Main Libraries Used:**
    - cSEM: A unified estimation engine for composite-based structural equation modeling.
    - genpathmox: The core algorithm for detecting unobserved heterogeneity via decision trees in PLS.

2. Requirements and Preparation

To ensure successful estimation, users must prepare their database following these guidelines:

- **File Format:** The application exclusively accepts Excel files (.xls, .xlsx).
- **Structure:** The first row must contain the names of the variables (indicators). The subsequent rows must be the cases or observations.
- **Data Types:** Indicators intended for the SEM model must be numeric. Segmentation variables for the PATHMOX module can be either categorical (text) or numeric.
- **Prior Considerations:** It is highly recommended to remove completely empty columns or duplicated column names before uploading the file to prevent parsing errors.

3. General View of the Interface

The application is organized using a **sidebar layout** alongside **main top navigation tabs**.

1. **Sidebar:** Contains critical controls that affect the entire application: project saving/loading, data import, missing data treatment, estimation algorithms, and resampling (Bootstrap) parameters.
2. **Main Panels (Tabs):** Organized sequentially from left to right (*Data -> Constructs -> Structural relations -> SEM results -> WPI -> PATHMOX*). *Recommendation:* Follow this flow strictly from left to right, as later estimations depend on the existence of valid data and constructs defined in earlier tabs.

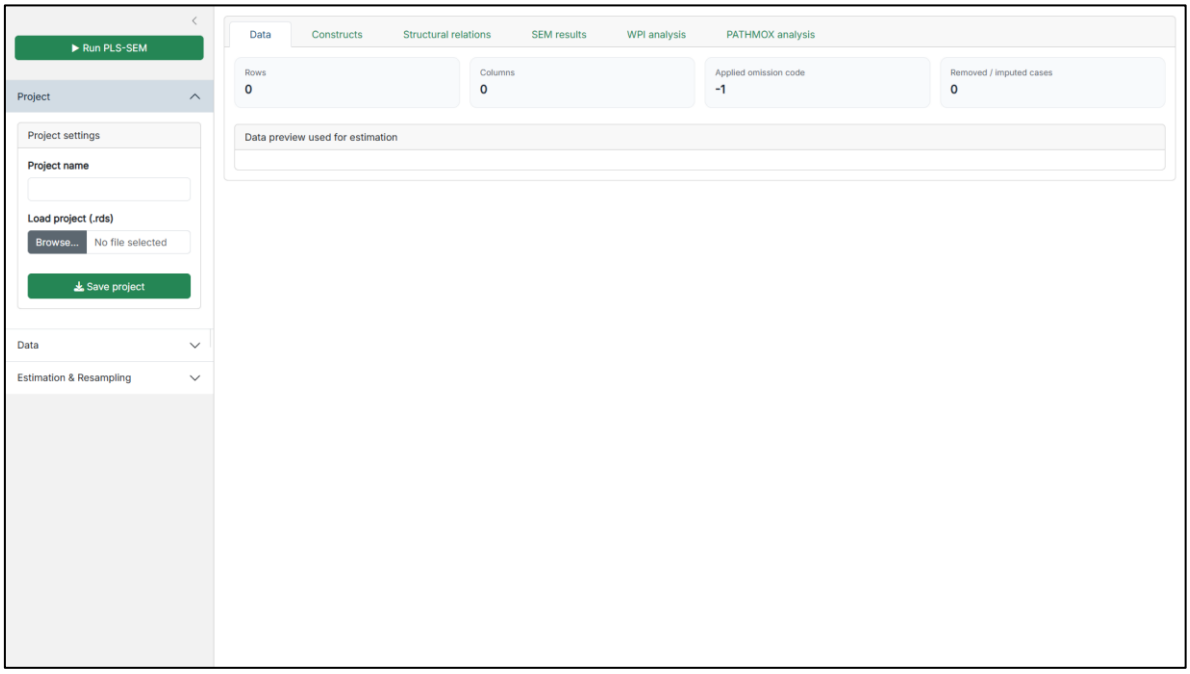

Figure 1. General view of the app

4. Step-by-Step Manual by Modules

4.1. Project Configuration and Data Import

Located in the sidebar and the **Data** tab.

- **Objective:** Load the database and handle missing values before estimation.
- **Data Upload:** Select your Excel file. When loaded, the app automatically clears previous selections to avoid data inconsistencies.
- **Code to be treated as missing:** Enter the numeric value that represents missing data in your survey (e.g., -1 or 999). The system will transform these into standard NA values.
- **Treatment of missing values:**
    - *Listwise deletion (recommended):* Removes any row containing at least one NA. Statistically safer if your sample size is large enough.
    - *Mean/Median imputation:* Replaces the NA with the mathematical average or median of that specific column. Valid only for numeric variables.
- **Outputs:** The Data panel shows "Rows", "Columns", the applied omission code, how many cases were removed/imputed, and a "Data preview" table.

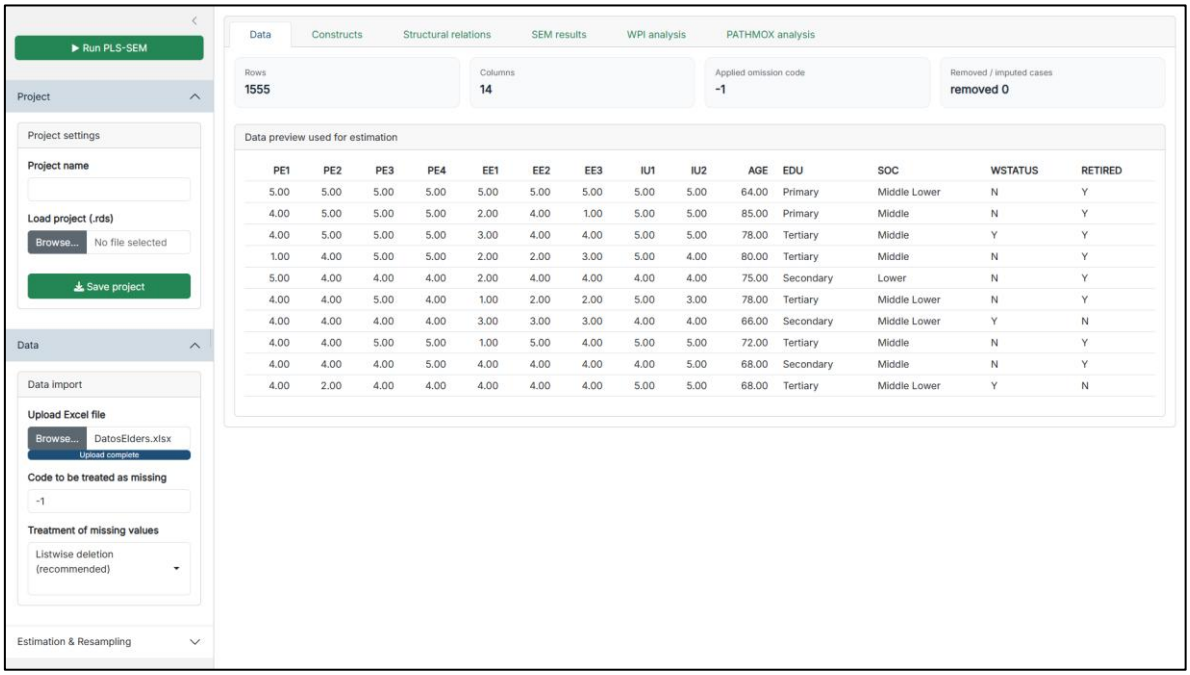

Figure 2. Data Module

4.2. Definition of Constructs (Measurement Model)

In the **Constructs** tab, the user defines which indicators measure which latent variable.

- **Objective:** Build the measurement model linking data columns to theoretical concepts.
- **Measurement Model Type:**
    - *Common factor:* The construct causes the indicators (Reflective model). Corresponds to the =~ operator in lavaan syntax.
    - *Composite:* The indicators form or create the construct (Formative model). Corresponds to the <~ operator.
- **UI Controls:** Type a name, select indicators from the "Available" list, and use the >> button to move them to "Assigned". Click "Save / update construct".
- **Restrictions:** An indicator assigned to one construct disappears from the "Available" list, preventing cross-loadings. If you delete a construct, its indicators become available again.

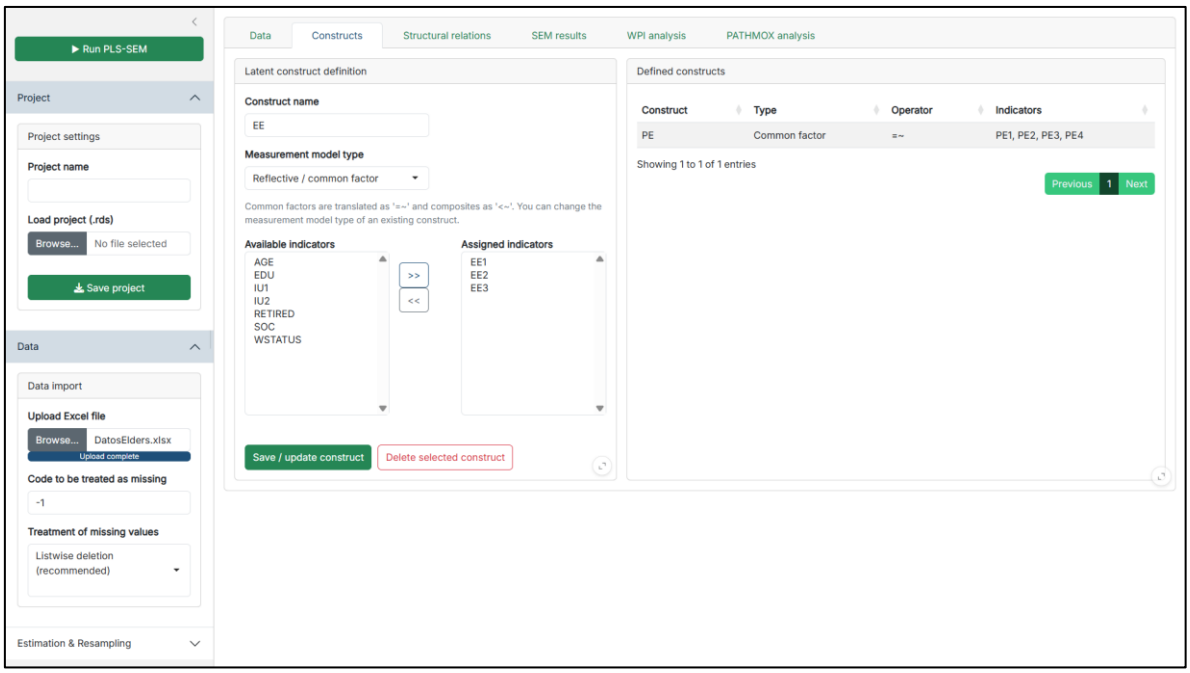

Figure 3. Constructs Editor

4.3. Structural Relations (Structural Model)

In the **Structural relations** tab, the user defines the research hypotheses.

- **Objective:** Specify the paths (causal links) between the previously created constructs.
- **Inputs:** Select a "Dependent variable" (target) and one or more "Independent variables" (origin).
- **Validations & Restrictions:**
  - The app automatically blocks a variable from predicting itself.
  - Reciprocal relationships (e.g., A -> B and B -> A simultaneously) and structural cycles (A -> B -> C -> A) are strictly prohibited. The UI will show an error notification ("Relation blocked (creates cycle)"), as PLS-SEM requires recursive models.
- **Outputs:** Displays a live "Model view" (path diagram) and the underlying "Model syntax" generated for the cSEM engine.

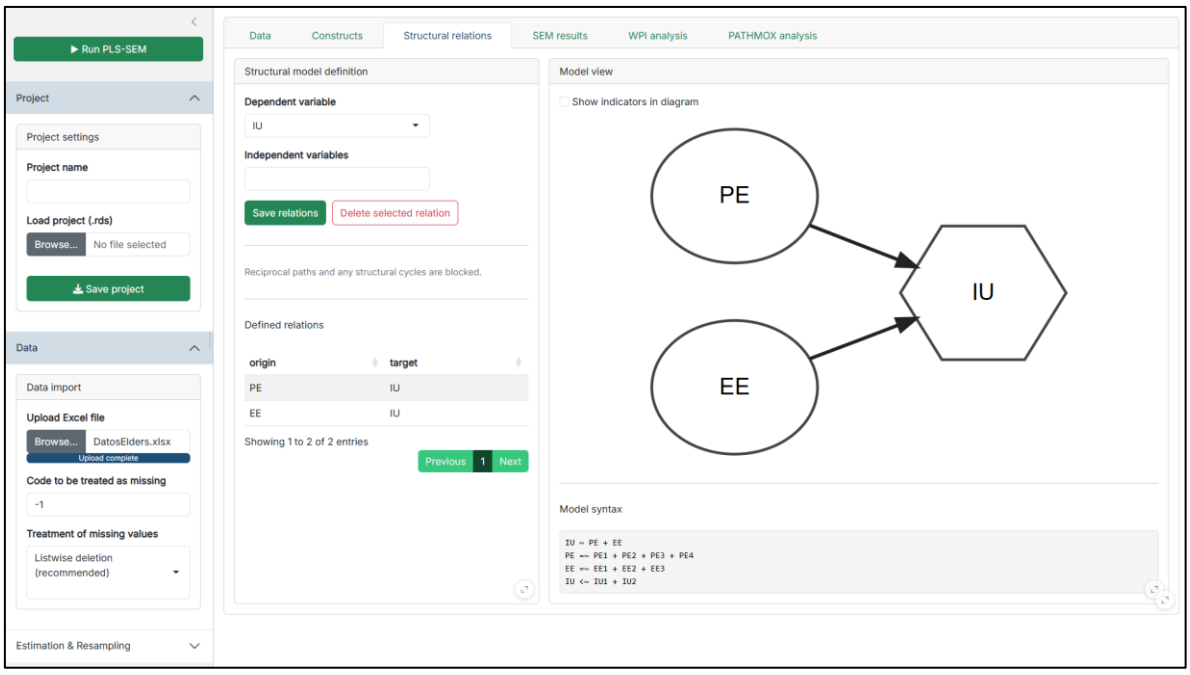

Figure 4. Structural Relations Editor and Model View

Additional built-in validation is performed when structural relations are saved. The application checks whether a proposed relation would generate an inadmissible recursive structure. A path is rejected if it creates self-prediction, reciprocity, or a structural cycle, helping ensure that the model passed to the estimation engine remains consistent with the recursive PLS-SEM workflow. The Structural Relations module should therefore be understood as a controlled model specification interface, not just as a visual editor.

4.4. SEM Estimation and Results

Once the model is configured, press the green **"▶ Run PLS-SEM"** button in the sidebar.

- **Objective:** Execute the statistical estimation and estimate the model and organize the main reporting outputs.
- **Outputs:** The application switches automatically to the **SEM results** tab.
- **Sub-panels (Pills):** Includes tables for Model fit, Construct types, R^2, Paths, Loadings, Measurement quality, HTMT, and Fornell-Larcker. It also renders the estimated path diagram showing R^2 values.
- **Export:** The "Export detailed Excel" button generates a multi-sheet .xlsx file containing the raw data, cleaned data, and all statistical tables.

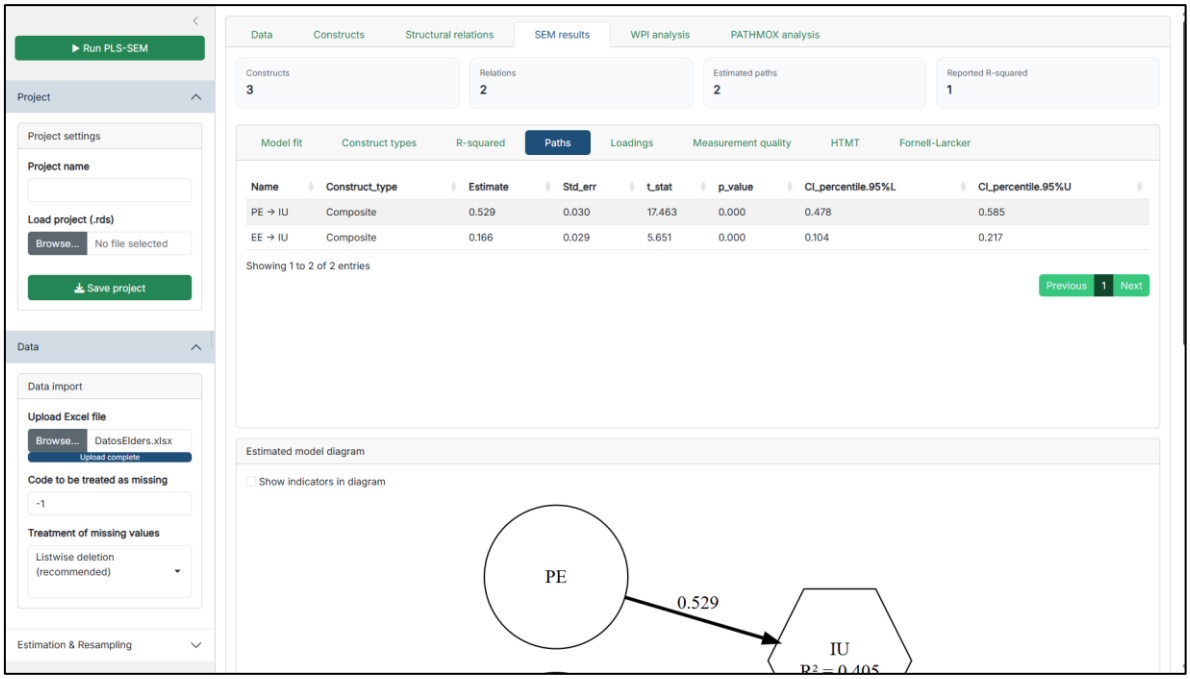

Figure 5. SEM Estimation and Results

In the implemented code, the SEM Results module retrieves and organizes outputs such as model fit, construct types, R^2, path coefficients, loadings, measurement quality, HTMT, and Fornell-Larcker results through dedicated extraction functions. The module also displays an estimated model diagram and a model status summary, allowing the user to move from a compact overview to more detailed inspection of the estimated solution. This organization is consistent with the role of cSEM as a unified framework for composite-based structural equation modeling, although the specific tables shown in the app depend on the extraction routines implemented in the Shiny project. Therefore, users should interpret the SEM Results tab as the application's structured reporting layer built on top of the cSEM estimation object.

4.5. WPI Analysis (Weighted Performance Index)

This advanced module acts as a post-estimation interpretive extension.

- **Objective:** Evaluates individual subject performance weighted by the PLS-SEM total effects.
- **Internal Logic:** The module starts from the already estimated SEM model and extracts the total effects associated with a user-selected Target Construct. It then rescales construct scores to a 0-100 range and computes a weighted performance index using the normalized absolute total effects of the predictors. In this sense, WPI is a post-estimation index derived from the model results rather than a standalone descriptive score.
- **Exploratory Segmentation Outputs:** Applies an automated K-Means clustering algorithm (calculating the optimal *k* via the elbow method) to group users into segments. Runs a t-SNE algorithm to project multi-dimensional performance into a 2D scatter plot. The module also stores individual-level and segment-level outputs, including WPI values, target scores, predictor performance values, t-SNE coordinates, and K-means segment membership. These results are then summarized in segment tables and individual tables, and can be exported as CSV data for further analysis. Although this logic is related to the importance-performance perspective (Importance Performance Map Analysis, IPMA), the implementation in this application is a custom weighted performance workflow based on total effects, score rescaling, t-SNE projection, and clustering. It should therefore be described as an application-specific post-estimation extension rather than as a direct replication of IPMA. Interpretation should proceed only after the SEM model has been judged acceptable at the measurement and structural levels. Because WPI depends on estimated total effects, weak or poorly specified models can produce misleading performance profiles.
- **Recommendation:** Use this to identify specific customer/user profiles (e.g., "High performers" vs. "Underperformers").

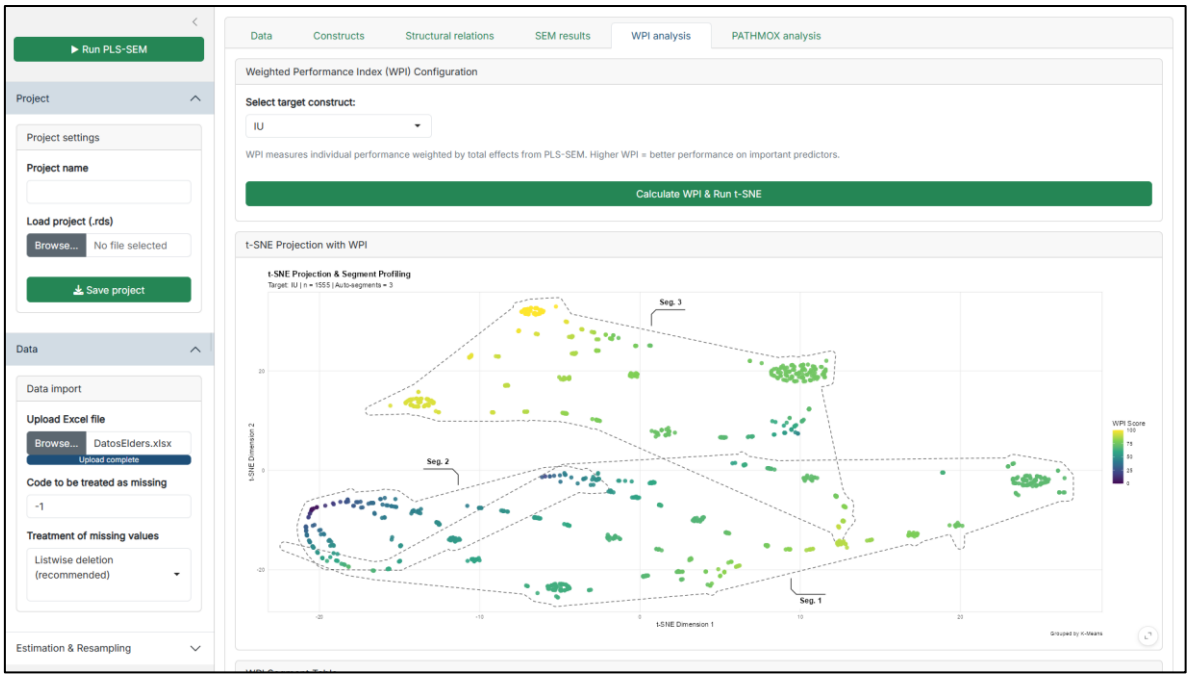

Figure 6. WPI Module

4.6. PATHMOX Analysis (Segmentation)

Designed for tree-based segmentation with additional MICOM and MGA-based group comparison outputs.

- **Objective:** Discover if the structural model changes significantly depending on certain variables (e.g., Gender, Income).
- **Inputs:** Allows selection of segmentation variables. Continuous variables can be discretized into quantiles or equal-width bins. In the implemented interface, segmentation variables can be configured as factors, kept as character variables, or discretized into quantiles or equal-width bins. When ordered categorical handling is used, the user can explicitly define and reorder levels before running the analysis. The module also allows the user to set alpha, maximum depth, minimum node size, minimum candidate split size, and bootstrap settings for MICOM and MGA-related procedures.
- **Advanced Feature (MICOM):** During the tree growth, the app integrates a Measurement Invariance (MICOM) check. If the new child nodes (e.g., Men vs. Women) do not interpret the survey questions identically (invariance fails), the split is rejected. This ensures mathematical validity. More precisely, the PATHMOX routine includes an optional invariance check during node splitting. If activated, the algorithm evaluates candidate child groups and can reject a split when invariance-related conditions fail, especially when the user chooses to stop the split if critical invariance is not met. This makes the segmentation workflow more restrictive and helps avoid retaining splits that are not sufficiently comparable across groups.
- **Outputs:** Renders the PATHMOX Tree, Terminal path comparisons, Variable importance, MICOM step 2 results, and Henseler MGA p-values. Beyond the global tree, the module also computes detailed SEM summaries for terminal nodes, including model fit, R^2, paths, loadings, measurement quality, HTMT, and Fornell-Larcker outputs for the selected terminal group. It also provides a terminal assignment table for individuals and a CSV download of the terminal-node data. As a result, PATHMOX in this application should be understood as both a segmentation tool and a group-specific SEM inspection workflow.

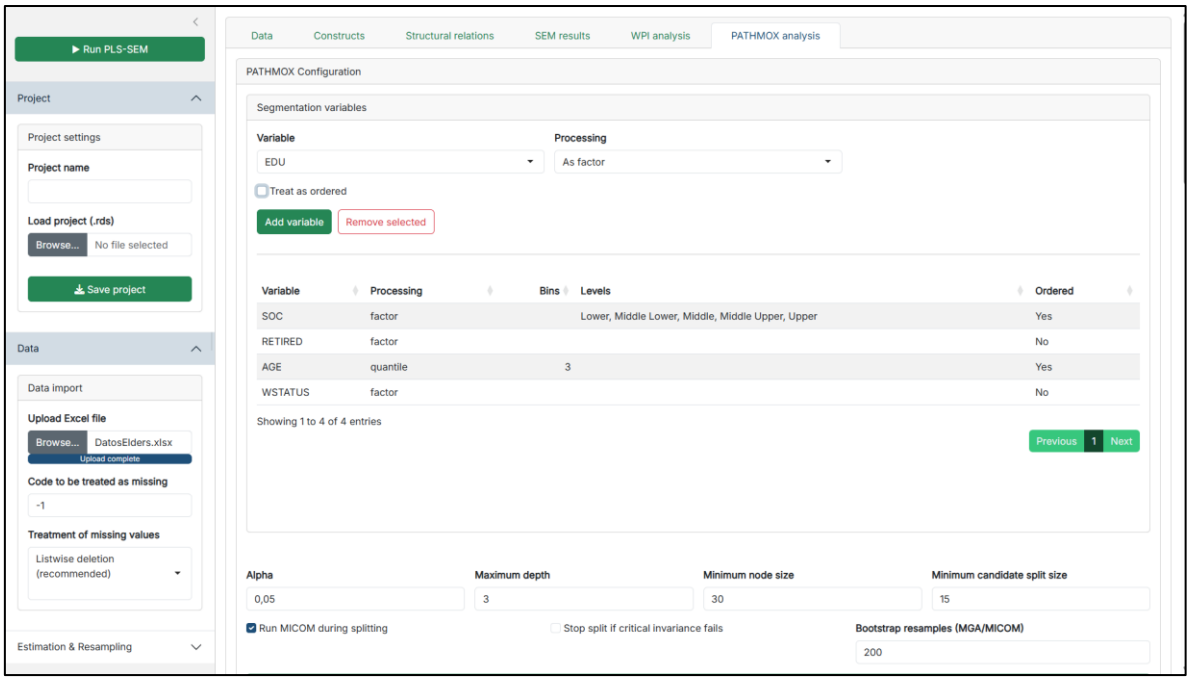

Figure 7a. PATHMOX Tree and MICOM tables

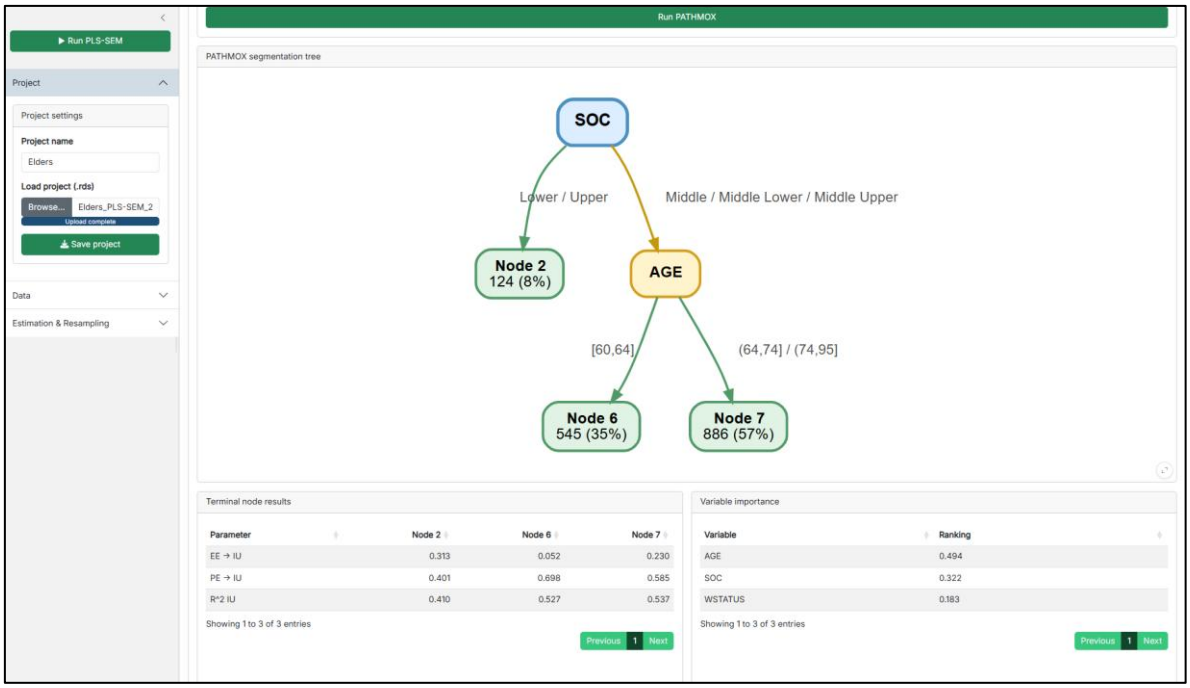

Figure 7b. PATHMOX Tree and MICOM tables

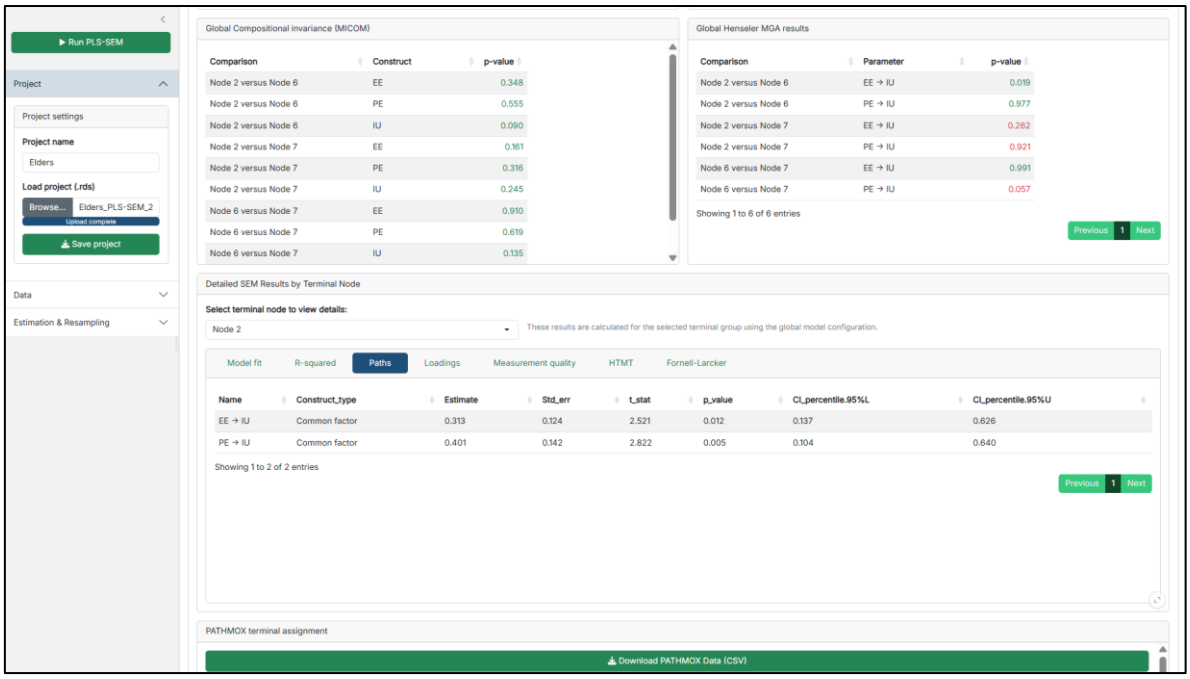

Figure 7c. PATHMOX Tree and MICOM tables

4.7. Save/Load Projects

- **Save project:** Downloads an .rds file containing your raw data, constructs, relations, results, and all sidebar settings.
- **Load project:** Upload a previously saved .rds file to instantly restore your entire session exactly as you left it.

5. Detailed Explanation of Estimation Parameters

This section details the sidebar controls, which map directly to the arguments of the cSEM::csem() function:

- **Weighting approach:** Indicates the approach used to estimate the model scores. Options include PLS-PM (default), GSCA, SUMCORR, ML, OLS, and 2SLS. cSEM presents these approaches within a unified composite-based SEM framework.
- **Path estimation:** Controls how the structural paths are estimated (OLS or 2SLS).
- **PLS inner weighting:** Defines the internal weighting scheme. *Path* (default) is recommended for structural models, alongside *centroid* and *factorial*.
- **Default outer mode:** Determines how measurement blocks are treated. *Auto* is highly recommended as it reads the construct type (Common factor = Mode A; Composite = Mode B).
- **Use PLSc correction for reflective constructs:** Activates the Dijkstra-Henseler correction. When concepts are modeled as common factors, PLS-PM inherently underestimates structural paths. This disattenuation corrects the attenuation bias.
- **Resample method:** Allows choosing between *bootstrap* (with replacement), *jackknife* (leave-one-out), or *none*. These methods are used to obtain standard errors and P-values for statistical inference.
- **Number of resamples:** The .R argument. The app proposes 200 for initial testing. For final publication, values between 500 and 10,000 are standard.
- **Handle inadmissibles:** Decides what to do with inadmissible bootstrap solutions (e.g., negative variances). *Replace* (default) repeats the resampling until a valid solution is found.

In practical terms, these controls are not merely descriptive labels in the interface. The implemented server logic passes the selected weighting approach, path estimation method, inner weighting scheme, resample method, number of resamples, inadmissible-solution handling, and PLSc disattenuation option directly to the cSEM::csem() estimation call. This means that changes made in the sidebar alter the actual estimation settings used by the model.

6. Results Interpretation Guide

For beginners, here is how to interpret the tables generated in the **SEM results** tab:

- **Model fit (SRMR):** Standardized Root Mean Square Residual. A value **< 0.08** generally indicates a good overall fit of the model.
- R^2 **(R-squared):** Represents the percentage of variance explained in the dependent variables. Values > 0.75 are substantial, around 0.50 are moderate, and 0.25 are weak.
- **Paths:** Look at the *Estimate* (the strength and direction of the effect, usually between -1 and 1) and the *P_value*. A **P-value < 0.05** means the hypothesis is statistically significant (supported).
- **Loadings:** Ideally, these should be **> 0.708**. This indicates that the indicator shares more than 50% of its variance with the latent construct.
- **Measurement quality (Reliability & Convergent Validity):** Cronbach's Alpha and rho_A should be **> 0.70**. AVE (Average Variance Extracted) must be **> 0.50**.
- **HTMT (Discriminant Validity):** Evaluates if constructs are truly distinct from one another. Values should strictly be **< 0.90** (ideally < 0.85).

Advanced outputs such as WPI and PATHMOX should be interpreted only after the global SEM model has been examined first. In this application, both modules depend on the validity of the estimated base model, because WPI uses total effects from the SEM results and PATHMOX applies segmentation and group comparison on top of the global model specification. For that reason, these advanced modules should be treated as complementary analytical layers, not as substitutes for measurement and structural assessment.

7. Troubleshooting and Common Errors

The application handles several errors internally. Here is how to resolve them:

1. **"Define constructs first" / "Please specify a construct name":** You attempted to run the model or save an empty construct. Ensure all text fields are filled and items are assigned.
2. **"Reciprocal relation blocked" / "Relation blocked (creates cycle)":** You tried to draw a path that creates a loop (e.g., A -> B -> A). Review your theoretical framework; PLS-SEM does not support feedback loops.
3. **App crashes or shows blank tree during PATHMOX:** Often caused by a segmentation variable with zero variance (e.g., 100% of respondents are "Female") or terminal nodes that are too small. Ensure your segmentation variables are balanced and increase the "Minimum node size" parameter.
4. **"Processed data still contains NAs":** If you chose "None" for missing data treatment, the cSEM algorithm will fail. Go back to Data Import and select "Listwise deletion" or "Imputation".

8. Best Practices Recommendations

- **Recommended Workflow Sequence:** Always follow the tabs from left to right: *Data -> Constructs -> Relations*. If you change the dataset, the app will erase the constructs to prevent data mapping errors.
- **Naming Conventions:** Avoid special characters, mathematical symbols, or spaces in your construct names (use Brand_Trust instead of Brand Trust!).
- **Methodological Precaution for PATHMOX:** Decision trees require large samples. It is recommended to have a total sample of at least 150-200 observations before running PATHMOX to ensure that the resulting segments maintain statistical power.
- **Prototyping vs. Final Run:** For exploratory learning, set the "Resample method" to *100* resamples. This makes estimation instantaneous. Once your model is validated, switch to *Bootstrap* with *5000+* resamples to get accurate P-values.

9. Installation and Deployment Guide

ShinyPLSp is distributed as an R package hosted directly on GitHub. Installing it is a straightforward process that can be done entirely within your R console.

Prerequisites: Before installing the application, ensure you have the following software installed on your computer:

- R: Download and install the latest version from CRAN.
- RStudio: Download and install RStudio Desktop from Posit.
- (For Windows Users Only): It is recommended to install *RTools* from CRAN, as compiling some dependencies might require it.

Step 1: Install the Package from GitHub

ince ShinyPLSp is hosted on GitHub, you will use the remotes package to install it. Open RStudio and run the following commands in your Console:

```
install.packages("remotes")
remotes::install_github("patricioramirezcorrea/ShinyPLSp")
```

Step 2: Launch the Application

Once the installation is complete, you can start the application using its main execution command. Run the following code in your RStudio Console:

```
library(ShinyPLSp)
run_app()
```

The ShinyPLSp application will automatically open in a new window or in your default web browser, ready for you to start your analysis.